PUBLIC PORTFOLIO TRANSLATION

Damavand Predictive AI Model Report

English Translation of Redacted Model Report

Document type: English translation of redacted Greek technical model report.

Project context: Industrial energy baseline modeling and Measurement & Verification case study.

Use: Public portfolio documentation for the Damavand Energy Forecasting Platform repository.

> **Note.** This is an English translation prepared for public portfolio review. The original Greek redacted report remains the authoritative case-study evidence. Detailed daily operational energy values remain redacted.

Damavand Predictive AI Model Report

English Translation - Public Redacted Portfolio Version

For correlating electricity consumption with influencing operational parameters and estimating electrical energy savings for the Damavand electrical installations.

Original report date: December 2025
Public translation purpose: case-study documentation for the Damavand Energy Forecasting Platform repository.

> This English version translates the redacted Greek report into a more readable public portfolio format. The original redacted Greek report remains the authoritative case-study document. Daily actual/predicted operational energy values remain redacted.

Contents

1. Introduction
2. Consumption Prediction Model with CatBoost
3. Data Cleaning
4. Model Training
5. Model Accuracy Validation
6. Electrical Energy Savings Result
7. Conclusions
8. References
9. Appendices
    - Appendix 1: Scientific Analysis of CatBoost
    - Appendix 2: Model Code
    - Appendix 3: Execution Results
    - Appendix 4: Dataset Synthesis and Feature Selection Methodology

---

1. Introduction

To estimate electrical energy savings at the Damavand facility, an artificial-intelligence model was trained to predict electricity consumption based on the parameters that affect it.

The parameters used by the model include the numerical representation of the date, calendar fields such as `year`, `month`, `weekday`, `is_weekend`, and `week_of_year`, and operational variables including `total_kg`, `total_nominal_kg`, `total_brix_units`, `total_hours`, `total_pallets`, `orders`, `avg_brix`, and `yield_ratio_actual_over_nominal`.

The model was trained using data from the period before commissioning of the energy-saving project, so that it could estimate the electricity consumption that would have occurred during the post-implementation period if the project had not been carried out.

By comparing the predicted counterfactual consumption with the actual measured consumption after installation of the equipment, the achieved electrical energy saving can be estimated.

No outlier-detection or outlier-removal process was applied, because it was considered important for the model to learn from real operating cases with strong variation in electricity consumption. These cases reflect actual operating conditions.

The model used was trained using the `CatBoostRegressor` method.

Based on the model results for predicting electricity consumption before the energy-saving project was commissioned, the training-period deviation was only `-0.46%`, confirming the accuracy of the prediction. The electrical energy saving achieved after commissioning of the energy-saving project was estimated at `11.89%`.

---

2. Consumption Prediction Model with CatBoost

A regression model trained with the CatBoost Regressor method was used for model training.

CatBoost, developed by Yandex, is a gradient-boosting framework based on ensembles of weak decision trees. At each training stage, a new tree is constructed to correct the errors of previous trees by predicting residual errors.

The predictions from the individual trees are combined in a weighted manner, controlled by hyperparameters such as learning rate, tree depth, and regularization.

CatBoost is known for high predictive accuracy and stability, as well as its ability to perform effectively on data with complex non-linear relationships. This makes it particularly suitable for applications involving electricity-consumption prediction.

Further technical details about CatBoost are provided in Appendix 1.

---

3. Data Cleaning

A common practice before training a predictive model is to detect and remove outliers in order to reduce their effect on the model training process.

In this analysis, however, the decision was made not to remove outliers. The reason is that these extreme points represent real and significant fluctuations in the electricity consumption of the Damavand facility and can provide useful information to the model.

Why outliers were not removed

1. **Real operating scenarios**
   Extreme electricity-consumption points in the Damavand case correspond to real operating conditions, such as emergency production increases, test operations, or peak-demand conditions.
2. **Information for the model**
   Retaining extreme examples allows the model to learn directly from those cases, improving its ability to generalize to similarly demanding scenarios.
3. **Minimization of bias**
   Removing such observations in advance could lead to loss of information or oversimplification of the dataset, causing undertraining on real electricity-consumption peaks.

---

4. Model Training

A Python script was developed to train the model using the `CatBoostRegressor` algorithm, a gradient-boosted tree model.

The model was trained using the following features:

- numerical representation of the date;
- calendar fields: `year`, `month`, `weekday`, `is_weekend`, `week_of_year`;
- operational variables: `total_kg`, `total_nominal_kg`, `total_brix_units`, `total_hours`, `total_pallets`, `orders`, `avg_brix`, and `yield_ratio_actual_over_nominal`.

The target variable was daily active electrical energy consumption:

```
active_energy_kWh
```

The `CatBoostRegressor` was trained with the following hyperparameters:

- `iterations = 185`
  The number of decision trees composing the model. A larger number increases learning capacity but also increases the risk of overfitting. The value of 185 iterations was selected as a balanced value, sufficient to learn data trends without excessive overfitting.
- `learning_rate = 0.18`
  Defines the step size with which the model learns from errors. The value 0.18 is considered moderate to high and suitable for the size and nature of the dataset.
- `depth = 6`
  The maximum depth of each decision tree. This allows the model to capture non-linear relationships while maintaining a satisfactory level of generalization.
- `loss_function = MAE`
  The cost function used for model optimization, emphasizing reduction of mean absolute deviation between predicted and actual values.
- `random_seed = 33`
  A fixed random seed used to ensure reproducibility of the results.

The final electricity-consumption prediction is produced by the CatBoostRegressor model, which was selected as the preferred solution after comparative evaluation.

The model combines strong predictive accuracy with robust generalization, making it appropriate for estimating daily energy consumption at the facility.

The selected hyperparameters were chosen to balance expressiveness and avoidance of overfitting. This configuration allows the model to capture complex relationships in the data while remaining stable and reproducible.

As in the data-cleaning stage, no outliers were removed. The model was allowed to learn from all real operating conditions, including production and consumption peaks.

Only records with zero active energy consumption (`active_energy_kWh = 0`) were excluded, because they do not represent real consumption and would introduce noise into the model.

In this way, the CatBoostRegressor provides reliable and realistic predictions of consumption while accounting for both typical and extreme operating conditions.

---

5. Model Accuracy Validation

Before proceeding to future predictions, the reliability of the CatBoostRegressor model was verified by comparing its estimates with actual data from the period before implementation of the electrical energy-saving project.

The evaluation was based on Mean Absolute Error (MAE) and percentage deviation.

The training period covered the period before the start of the reporting period, excluding the independent validation period from `28/11/2024` to `05/12/2024` , which was used exclusively for validation.

During the training period, the model achieved:

- `MAE = 544.67 kWh`
- `Deviation = -0.46%`

This confirmed accurate learning behavior.

During the validation period from `28/11/2024` to `05/12/2024` , the model also showed a low percentage deviation between predicted and actual consumption. This confirms its ability to generalize and produce reliable predictions on data not used during training.

Validation-period results were:

- `MAE = 3035.28 kWh`
- `Deviation = -2.24%`

The negative deviation means the model does not overestimate consumption. This strengthens the credibility of the energy-savings estimate by avoiding artificial inflation of savings.

Finally, the reporting period begins on `12/09/2025` and is used to estimate electricity savings by comparing model-predicted baseline consumption with actual post-intervention consumption.

Redacted daily operational tables

The original report included daily actual and predicted values for training, validation, and reporting periods.

Those detailed daily operational values have been removed from the public portfolio version because they may reveal commercially sensitive operating patterns.

Aggregate metrics and methodology are retained.

---

6. Electrical Energy Savings Result

After training and evaluating the electricity-consumption prediction model using the CatBoostRegressor algorithm, it became possible to estimate the savings achieved after commissioning of the SENERQON project at Damavand.

The model was applied to predict daily electricity consumption for the period after the start of the reporting period, beginning on `12/09/2025`. This is the period when the energy interventions were already fully operational.

Because the model was trained exclusively on data from before project implementation, its predictions represent the level of consumption that would likely have occurred if the energy-saving interventions had not been implemented.

The comparison between these theoretical baseline predictions and the actual recorded consumption during the same period, which shows reduced values due to operation of the project, allows the energy saving to be estimated quantitatively.

The difference between predicted and actual consumption provides objective evidence of the effectiveness of the interventions and supports confidence in the reliability of the model and the accuracy of the results.

Detailed daily reporting-period actual/predicted values were redacted for the public portfolio version.

As shown by the aggregate result, for the period after commissioning of the SENERQON project, the estimated electricity saving was:

```
11.89%
```

The project target was at least `10.1%` savings. The final result exceeded expectations and confirmed that the energy-saving project was successful.

The use of CatBoostRegressor, trained exclusively on pre-intervention consumption data, provides an objective estimate of savings, provided that factory production activity remains comparable with the training period.

If major operational changes occur, such as the addition of new production lines, the model structure and data may no longer be sufficient for reliable prediction of theoretical consumption. In such cases, the energy consumption of the new production line should be accounted for separately and removed from the calculation results where appropriate.

---

7. Conclusions

The full set of historical data provided by Damavand was used without removing outliers, allowing the model to account for real peak operating conditions in electricity consumption.

A CatBoostRegressor electricity-consumption prediction model was trained on this complete and representative dataset.

The model uses both calendar features and operational variables:

- calendar features: `year`, `month`, `weekday`, `is_weekend`, `week_of_year`;
- operational variables: `total_kg`, `total_nominal_kg`, `total_brix_units`, `total_hours`, `total_pallets`, `orders`, `avg_brix`, `yield_ratio_actual_over_nominal`.

The training period covered the interval before `01/06/2024` through `11/09/2025`, before commissioning of the energy-saving project.

During training, the model produced very low Mean Absolute Error and percentage deviation:

- `MAE = 544.67 kWh`
- `Deviation = -0.46%`

For generalization evaluation, an independent validation period was selected from `28/11/2024` to `05/12/2024`. During this period the model also showed low deviation between predicted and actual consumption:

- `MAE = 3035.28 kWh`
- `Deviation = -2.24%`

This result confirms that the model does not show overfitting and can generalize reliably to data not used during training.

The negative deviation in the validation period means the model predicted lower consumption than actual consumption. This is important because it makes the model conservative with respect to savings: the savings calculated in practice are not inflated by an overestimated baseline.

During the post-implementation period from `12/09/2025` to `31/10/2025`, when the SENERQON energy-saving interventions were fully operational, actual daily consumption was compared with the model's theoretical predictions under the no-intervention scenario.

The estimated average electrical energy saving was `11.89%`, clearly exceeding the original project target of `10.1%`.

These results highlight the success of the energy-saving project and confirm that the developed model is a reliable tool for supporting and validating the agreed energy-saving result.

---

8. References

1. Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). *CatBoost: Unbiased Boosting with Categorical Features*. Advances in Neural Information Processing Systems (NeurIPS), 31, 6638-6648. https://proceedings.neurips.cc/paper/2018/file/14491b756b3a51daac41c24863285549-Paper.pdf

2. Dorogush, A. V., Ershov, V., & Gulin, A. (2018). *CatBoost: Gradient Boosting with Categorical Features Support*. arXiv:1810.11363. https://arxiv.org/abs/1810.11363
3. CatBoost Documentation. *CatBoost: Open-Source Gradient Boosting Library*. https://catboost.ai/docs/
4. CatBoost Documentation. *CatBoostRegressor Parameters*. https://catboost.ai/docs/concepts/python-reference_catboostregressor.html
5. Scikit-Learn. *Gradient Boosting Regressor - General Concepts*. https://scikit-learn.org/stable/modules/ensemble.html#gradient-boosting
6. Wikipedia. *CatBoost*. https://en.wikipedia.org/wiki/CatBoost
7. Wikipedia. *Gradient boosting*. https://en.wikipedia.org/wiki/Gradient_boosting
8. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer. Chapter 10: Boosting and Additive Trees. https://web.stanford.edu/~hastie/ElemStatLearn/
9. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 6: Feedforward Deep Networks. https://www.deeplearningbook.org/
10. Scikit-Learn. *sklearn.neural_network.MLPRegressor*. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html
11. Wikipedia. *Multilayer perceptron*. https://en.wikipedia.org/wiki/Multilayer_perceptron
12. Gardner, M. W., & Dorling, S. R. (1998). *Artificial neural networks (the multilayer perceptron) - A review of applications in the atmospheric sciences*. Atmospheric Environment, 32(14-15), 2627-2636. https://doi.org/10.1016/S1352-2310(97)00447-0
13. Zhang, G., Eddy Patuwo, B., & Hu, M. Y. (1998). *Forecasting with artificial neural networks: The state of the art*. International Journal of Forecasting, 14(1), 35-62. https://doi.org/10.1016/S0169-2070(97)00044-7
14. Baldi, P. (2012). *Autoencoders, unsupervised learning, and deep architectures*. Proceedings of ICML Workshop on Unsupervised and Transfer Learning, PMLR, 37-49. http://proceedings.mlr.press/v27/baldi12a/baldi12a.pdf

9. Appendices

Appendix 1. Scientific analysis of CatBoost

CatBoost, or Categorical Boosting, is an advanced gradient-boosting algorithm designed for high accuracy, stability, and effective generalization in regression problems.

It follows a step-by-step process in which sequential decision trees, or weak learners, are trained on the errors of previous trees. This gradually improves prediction quality and combines the results into a weighted sum.

CatBoost is based on the logic of gradient boosting, while incorporating advanced techniques to reduce bias and overfitting as well as internal mechanisms for regularization.

1. Initialization

The model starts with an initial prediction for each sample, usually the mean value of the target variable, such as average daily electricity consumption.

2. Residual calculation

For each observation, residuals are calculated as the difference between the actual value and the current prediction.

These residuals represent the errors that the next decision tree is trained to correct.

3. Training a new weak tree

A new, relatively shallow decision tree is trained to approximate the residuals.

Each such tree acts as a weak learner and has limited depth, for example `depth = 4-8`, to avoid overfitting and improve generalization.

4. Prediction update

The prediction from the new tree is added to the previous prediction using the learning-rate coefficient `η`:

$$\hat{y}_t = \hat{y}_{t-1} + \eta \cdot f_t(x)$$

where `f_t(x)` is the prediction function of the current tree and `η` (eta) is the learning rate. The learning rate controls the influence of each new tree and enables gradual, stable model improvement.

5. Regularization and complexity control

CatBoost includes mechanisms for controlling tree complexity and avoiding overfitting by limiting tree depth, branch structure, and learning progress.

These mechanisms support model stability even in noisy data or data with strong variation.

6. Repetition and combination

Steps 2-5 are repeated for `N` iterations.

At the end of the process, the final prediction is produced as the sum of all individual corrections.

This weighted sum ensures that more accurate learners contribute more strongly to the final result.

Main advantages of CatBoost

1. Increased accuracy through boosting

CatBoost combines many weak learners sequentially, allowing the final model to achieve high predictive accuracy.

For daily energy-consumption prediction, this is critical for reliably representing both normal and changing operating conditions.

2. Focus on difficult cases

The model adjusts its training by placing more emphasis on observations with larger prediction errors.

This gradually improves performance on complex patterns, seasonal changes, and non-linear consumption behavior.

3. Stability and overfitting control

CatBoost is designed to reduce bias and overfitting through internal regularization and controlled learning.

This is especially important in applications with real-world data, where peaks and noise exist.

4. High performance and computational efficiency

CatBoost is implemented in C++, supporting high computational efficiency and fast training. This makes it suitable for industrial applications and large datasets.

5. Flexibility and parameterization

CatBoost provides flexibility through hyperparameters such as number of iterations, tree depth, and learning rate. This allows the model to be adapted to the application and the available data.

Practical applications of CatBoost

1. Time-series forecasting

CatBoost is widely used in time-series forecasting problems, such as energy consumption, product demand, and financial indicators.

In this work, it is used to predict daily electricity consumption using both calendar and operational features.

2. Industrial monitoring and predictive maintenance

CatBoost can be used for anomaly detection, equipment-failure prediction, and industrial-process optimization, especially in environments with continuous data flows.

Appendix 1 conclusion

The CatBoostRegressor algorithm proved highly effective in predicting daily electricity consumption for Damavand.

Through sequential training of weak trees that correct previous errors, the model was able to capture both normal variation and real extreme consumption values without losing critical information.

Its adaptive nature, combined with stability and computational efficiency, makes it particularly suitable for industrial applications requiring reliable and repeatable prediction.

For the Damavand case, applying the model led to very low deviation and a valid estimate of the achieved consumption reduction, confirming CatBoost as a strong and practical tool for energy-demand prediction.

Appendix 2. Model code

The model was written in Python and can be run on any platform. It takes as input the final dataset created from the files provided by the client. The execution results are shown in the following appendix.

```
import pandas as pd
import numpy as np
import tkinter as tk
from tkinter import filedialog
from catboost import CatBoostRegressor
from sklearn.metrics import mean_absolute_error

# ==========================
# CONFIG
# ==========================
DATE_COL = "Ημ/νία"
TARGET = "active_energy_kWh"

TRAIN_END = pd.to_datetime("2025-09-11")
TEST_START = pd.to_datetime("2025-09-12")

# VALIDATION PERIOD (INSIDE BASELINE)
VAL_START = pd.to_datetime("2024-11-28")
VAL_END = pd.to_datetime("2024-12-05")

OUT_TRAIN_XLSX = "Training_Period.xlsx"
OUT_VAL_XLSX = "Validation_Period.xlsx"
OUT_TEST_XLSX = "Prediction_Period.xlsx"

# ==========================
# FILE PICKER
# ==========================
root = tk.Tk()
root.withdraw()

file_path = filedialog.askopenfilename(
    title="Select dataset (CSV or Excel)",
    filetypes=[("CSV files", "*.csv"), ("Excel files", "*.xlsx")]
)

if not file_path:
    raise RuntimeError("No file selected")

# ==========================
# LOAD DATA
# ==========================
if file_path.lower().endswith(".csv"):
    df = pd.read_csv(file_path, sep=";", decimal=",", encoding="utf-8-sig")
else:
    df = pd.read_excel(file_path)


def parse_dates_robust(s):
    d1 = pd.to_datetime(s, dayfirst=True, errors="coerce")
    d2 = pd.to_datetime(s, dayfirst=False, errors="coerce")
    return d1 if d1.notna().sum() >= d2.notna().sum() else d2
```

```
df[DATE_COL] = parse_dates_robust(df[DATE_COL])
df = df.dropna(subset=[DATE_COL, TARGET]).copy()

# ==========================
# FEATURES
# ==========================
FEATURES = [
    "total_kg",
    "total_nominal_kg",
    "total_brix_units",
    "total_hours",
    "total_pallets",
    "orders",
    "avg_brix",
    "yield_ratio_actual_over_nominal",
    "weekday",
    "is_weekend",
    "month",
    "year",
    "week_of_year",
]

# ==========================
# SPLITS
# ==========================
train_mask = (
    (df[DATE_COL] <= TRAIN_END) &
    ~((df[DATE_COL] >= VAL_START) & (df[DATE_COL] <= VAL_END))
)
val_mask = (df[DATE_COL] >= VAL_START) & (df[DATE_COL] <= VAL_END)
test_mask = df[DATE_COL] >= TEST_START

X_train = df.loc[train_mask, FEATURES]
y_train = df.loc[train_mask, TARGET]

X_val = df.loc[val_mask, FEATURES]
y_val = df.loc[val_mask, TARGET]

X_test = df.loc[test_mask, FEATURES]
y_test = df.loc[test_mask, TARGET]

# ==========================
# MODEL
# ==========================
model = CatBoostRegressor(
    iterations=185,
    learning_rate=0.18,
    depth=6,
    loss_function="MAE",
```

```
    random_seed=33,
    verbose=False,
    allow_writing_files=False,
)

model.fit(X_train, y_train)

# =========================
# PREDICTIONS
# =========================
pred_train = model.predict(X_train)
pred_val = model.predict(X_val)
pred_test = model.predict(X_test) if len(X_test) else np.array([])

# =========================
# METRICS
# =========================
def totals_deviation_pct(y_true, y_pred):
    return (np.sum(y_pred) - np.sum(y_true)) / np.sum(y_true) * 100


train_mae = mean_absolute_error(y_train, pred_train)
val_mae = mean_absolute_error(y_val, pred_val)
test_mae = mean_absolute_error(y_test, pred_test) if len(y_test) else np.nan

train_dev = totals_deviation_pct(y_train, pred_train)
val_dev = totals_deviation_pct(y_val, pred_val)
test_dev = totals_deviation_pct(y_test, pred_test) if len(y_test) else np.nan

# =========================
# EXPORT EXCELS
# =========================
train_df = pd.DataFrame({
    "Date": df.loc[train_mask, DATE_COL].dt.strftime("%d/%m/%Y"),
    "Actual": y_train.astype(float).values,
    "Predicted": pred_train.astype(float),
})

train_footer = pd.DataFrame([
    {"Date": "", "Actual": np.nan, "Predicted": np.nan},
    {"Date": "MAE:", "Actual": train_mae, "Predicted": np.nan},
    {"Date": "Deviation (%):", "Actual": train_dev, "Predicted": np.nan},
])

pd.concat([train_df, train_footer], ignore_index=True).to_excel(OUT_TRAIN_XLSX,
index=False)

val_df = pd.DataFrame({
    "Date": df.loc[val_mask, DATE_COL].dt.strftime("%d/%m/%Y"),
    "Actual": y_val.astype(float).values,
    "Predicted": pred_val.astype(float),
```

```
})

val_footer = pd.DataFrame([
    {"Date": "", "Actual": np.nan, "Predicted": np.nan},
    {"Date": "MAE:", "Actual": val_mae, "Predicted": np.nan},
    {"Date": "Deviation (%):", "Actual": val_dev, "Predicted": np.nan},
])

pd.concat([val_df, val_footer], ignore_index=True).to_excel(OUT_VAL_XLSX, index=Fal
se)

test_df = pd.DataFrame({
    "Date": df.loc[test_mask, DATE_COL].dt.strftime("%d/%m/%Y"),
    "Actual": y_test.astype(float).values,
    "Predicted": pred_test.astype(float) if len(pred_test) else np.array([],
dtype=float),
})

test_footer = pd.DataFrame([
    {"Date": "", "Actual": np.nan, "Predicted": np.nan},
    {"Date": "MAE:", "Actual": test_mae, "Predicted": np.nan},
    {"Date": "Deviation (%):", "Actual": test_dev, "Predicted": np.nan},
])

pd.concat([test_df, test_footer], ignore_index=True).to_excel(OUT_TEST_XLSX,
index=False)

# ==========================
# CONSOLE OUTPUT
# ==========================
print("\n===== BASELINE TRAINING =====")
print(f"MAE: {train_mae:.2f} kWh")
print(f"Deviation: {train_dev:.2f}%")

print("\n===== BASELINE VALIDATION (28/11/24 - 05/12/24) =====")
print(f"MAE: {val_mae:.2f} kWh")
print(f"Deviation: {val_dev:.2f}%")

print("\n===== REPORTING / SAVINGS PERIOD =====")
print(f"MAE: {test_mae:.2f} kWh")
print(f"Deviation (Savings): {test_dev:.2f}%")

print("\nFiles saved:")
print(f"- {OUT_TRAIN_XLSX}")
print(f"- {OUT_VAL_XLSX}")
print(f"- {OUT_TEST_XLSX}")
```

Appendix 3. Code execution results

The code produced the following aggregate output:

```
===== BASELINE TRAINING =====
MAE: 544.67 kWh
Deviation: -0.46%

===== BASELINE VALIDATION (28/11/24 - 05/12/24) =====
MAE: 3035.28 kWh
Deviation: -2.24%

===== REPORTING / SAVINGS PERIOD =====
MAE: 5303.39 kWh
Deviation (Savings): 11.89%

Files saved:
- Training_Period.xlsx
- Validation_Period.xlsx
- Prediction_Period.xlsx
```

Appendix 4. Dataset synthesis and feature-selection methodology

1. General approach

The objective of the analysis was to create a unified daily dataset that accurately describes:

- the operational and production activity of the factory; and
- its relationship with daily active electricity consumption.

The dataset needed to be:

- temporally aligned with energy data;
- operationally interpretable;
- suitable for baseline model training.

2. Initial dataset and scale mismatch

The client's initial dataset contained production/order-level data, where each row corresponded to:

- a specific production order;
- a specific product or batch;
- quantities, BRIX, production hours, packaging type, and similar fields.

In contrast, electricity-consumption data was available on a daily basis.

This created the main issue:

```
It is not possible to directly correlate individual production rows with daily
electricity consumption.
```

3. Daily aggregation

To solve this issue, aggregation was applied.

All production rows belonging to the same date were combined into a single record per day.

Each day was treated as one operational unit describing:

- how much was produced;
- with what intensity;
- with what complexity;
- and under which time/calendar context.

This aggregation is necessary and consistent with international practice in model-based energy baselines.

4. Description of final features

4.1 `total_kg`

Definition: total actual kilograms produced per day.

Role in the model: main indicator of production load. In industrial processes, energy consumption is directly related to the total mass processed by the system.

4.2 `total_nominal_kg`

Definition: theoretical or nominal production kilograms according to specifications.

Role: not used as an independent consumption driver alone, but useful for performance checking and calculation of the yield indicator.

4.3 `yield_ratio_actual_over_nominal`

Definition: ratio of actual production to nominal production.

```
yield = actual kg / nominal kg
```

Role: captures the operational efficiency of the day. Lower yield is often associated with more losses, increased operating time, and therefore higher consumption.

4.4 `total_hours`

Definition: total production hours, excluding washing time.

Role: acts as a proxy for machine operating time, motor operation, and auxiliary loads. It captures the duration of energy demand.

4.5 `total_pallets`

Definition: total number of pallets produced during the day.

Role: represents packaging and logistics activity, related to forklifts, transport systems, and auxiliary consumption.

4.6 `orders`

Definition: number of different production orders per day.

Role: captures daily operational complexity. More orders imply more changeovers, more frequent adjustments, and less stable operation.

4.7 `total_brix_units`

Definition: total BRIX units processed during the day.

Role: captures process intensity, not just quantity. Higher BRIX implies increased thermal and mechanical energy demand.

4.8 `avg_brix`, weighted average

Definition: weighted average BRIX based on produced kilograms.

$$\text{avg\_brix} = \Sigma(kg_i \cdot brix_i) / \Sigma kg_i$$

Role: represents the average product of the day while avoiding distortion from small batches.

4.9 Calendar features

The calendar features used were:

- `weekday`
- `is_weekend`
- `month`
- `year`
- `week_of_year`

Role: these features capture operating patterns, seasonality, and differences between working and non-working days.

5. Features examined but rejected

During the analysis, variables such as the following were also considered:

- product code;
- product description;
- order code;

- detailed packaging categories;
- individual production phases.

They were rejected for the following reasons.

1. They are not uniquely defined at daily level

Multiple products and phases may coexist on the same day.

2. High dimensionality without physical interpretation

They can introduce noise without adding stable information.

3. Increased risk of overfitting

The model may learn codes instead of physical behavior.

4. Difficulty of presentation and technical review

These variables are harder to explain and justify consistently during technical review because they are not stable daily-level physical drivers.

Their information is not entirely lost. It is indirectly represented through:

- `total_kg`
- `total_hours`
- `orders`
- BRIX-related metrics.

6. Final result

The final dataset:

- describes each day using physical, measurable quantities;
- is temporally aligned with energy consumption;
- does not use any energy variable as input;
- provides a reliable basis for a model-based baseline.

This structure enables reliable model training and documented calculation of savings after interventions.

7. Conclusion

The dataset synthesis process was not a simple data summary.

It was a structured representation of the factory's real operation.

By converting raw production/order-level data into a daily model-ready dataset, the project created an operationally meaningful and machine-learning-ready foundation for baseline modeling.

This data-preparation work is a major part of the technical value of the project.

End of English translation.